Technical Infrastructure Documentation

Max Jansen
Tom Renardel De Lavalette
Pierre Mulder
Berkay Karayakuplu
Sohaib Kamri

June 22, 2026

Contents

1 Introduction

Within UMC Utrecht, research is being conducted into the use of artificial intelligence to support radiological work. This is relevant because the workload in radiology is high, and the assessment of medical images takes a lot of time. AI models can help make parts of this process more efficient, so radiologists can focus more on complex assessments and less on repetitive or time-consuming work.

At the moment, AI models are often developed and trained by individual researchers with specific technical knowledge. A central environment where researchers can easily develop, train, and repeat experiments is missing. As a result, much of the process is still manual. Researchers who want to work on AI models can lose valuable time setting up environments, managing dependencies, and preparing infrastructure instead of focusing on the research itself.

To address this problem, this proof-of-concept environment provides a standardized and automated way to train AI models. A researcher places code in a GitLab repository, selects a shared deployment template, and pushes the project. GitLab then builds the container image, RKE2 runs the workload on Kubernetes, and the workload uses shared SMB storage for input data, output files, logs, and executed notebooks.

The environment consists of three main systems. GitLab is the central development platform. It stores user projects and reusable templates, starts CI/CD pipelines, provides the container registry, and uses GitLab Runner to perform the build and deployment steps. RKE2 provides the Kubernetes cluster where the training workloads run. The SMB server provides shared storage so datasets, models, results, and logs can be stored centrally instead of on local machines.

The templates repository is important because it keeps the workflow accessible for researchers. Without it, every user project would need its own pipeline, Docker build logic, and Kubernetes manifests. In this setup, the reusable parts are stored centrally. A user project only includes the template it needs. This makes the environment easier to use, keeps project repositories small, and makes it possible to support different AI frameworks and workflows without rebuilding the infrastructure for every project.

This document explains how the technical infrastructure for this proof of concept is installed and used. It covers the recommended components, the installation of GitLab, RKE2, and SMB, the configuration that connects those systems, the templates repository, and the user workflow for running workloads. The document does not define a production architecture, high-availability design, backup strategy, monitoring platform, or full security policy. Those topics must be designed separately before the environment is used for production workloads or sensitive operational data.

2 Components

2.1 Recommended Setup

The infrastructure can run on a virtualized or cloud-based platform, as long as it provides the required compute, storage, and GPU resources to the virtual machines. Cloud infrastructure was used while developing this proof of concept.

The specifications below are meant for testing and validation. They are not production sizing advice. A production environment should be sized separately, based on the number of users, the expected workloads, storage growth, backup requirements, availability requirements, and security policy.

2.2 GitLab

The GitLab virtual machine is the central platform in this environment. It stores the repositories, manages CI/CD configuration, integrates with GitLab Runner, and provides the container registry used by the deployment workflow. Project access, deploy tokens, and pipeline settings are also managed here. In this proof of concept, GitLab is sized for a small test environment.

Recommended specifications

- OS: Ubuntu 24.04.4 (Noble Numbat)
- HDD: 256 GB
- CPU: 4vCPU
- RAM: 16 GB

2.3 RKE2

The RKE2 virtual machine provides the Kubernetes environment. It runs the application workloads, connects to the GitLab container registry, and provides the cluster services needed to deploy and test containers. This machine also has access to the NVIDIA GPU, so GPU-enabled jobs can be scheduled inside the cluster.

Recommended specifications

- OS: Ubuntu 24.04.4 (Noble Numbat)
- HDD: 256 GB
- CPU: 4vCPU
- RAM: 16 GB
- GPU: NVIDIA

2.4 SMB

The SMB virtual machine provides shared file storage. It exposes a central network share that can be reached by the other systems and by workloads running in Kubernetes. This makes it possible to test shared data access, persistent output, and storage integration between the infrastructure components.

Recommended specifications

- OS: Ubuntu 24.04.4 (Noble Numbat)
- SSD: 128 GB
- CPU: 2vCPU
- RAM: 2 GB

3 Installation

This section walks through the installation of GitLab, RKE2, and SMB. The steps are grouped by server and component, so it stays clear where each command must be executed. Follow the subsections in order. GitLab is prepared first, the Kubernetes server follows, and the SMB server is configured after the supporting services are in place.

During the installation, the operating systems are updated, required services are enabled, platform packages are installed, and access between the components is configured. Replace all values between angle brackets with values from the target environment before running the commands. At the end of this section, the servers should be ready for GitLab hosting, Kubernetes workloads, and shared storage access.

3.1 GitLab

3.1.1 Base Configuration, SSH & Firewall

Start on the GitLab server by updating the package index and installing the available upgrades. This gives the rest of the installation a current system base.

```
sudo apt update && sudo apt upgrade -y
```

Enable and start SSH so administrators can manage the GitLab server remotely.

```
sudo systemctl enable --now ssh.service
```

Open the ports needed for SSH, HTTP, HTTPS, and the GitLab container registry. After adding the rules, enable the firewall so they become active.

```
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw allow 5050/tcp
sudo ufw enable
```

3.1.2 Docker

Reference: `https://docs.docker.com/engine/install/ubuntu/`

Before Docker is installed from the official Docker repository, remove older or conflicting Docker-related packages from the Ubuntu repositories. This prevents package conflicts during installation.

```
sudo apt remove $(dpkg --get-selections docker.io docker-compose docker-
    compose-v2 docker-doc podman-docker containerd runc | cut -f1)
```

Add Docker's official signing key and package repository. After that, refresh the package index so Ubuntu can find the Docker packages from this repository.

```
# Add Docker's official GPG key:
sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/
    keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Add the repository to Apt sources:
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
```

Install Docker Engine, the Docker CLI, containerd, and the build and compose plugins. Later in the setup, GitLab Runner uses Docker as its executor.

```
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-
    plugin docker-compose-plugin
```

Enable and start Docker so it is available immediately and starts again after a reboot.

```
sudo systemctl enable --now docker.service
```

3.1.3 GitLab CE

Reference: `https://docs.gitlab.com/install/package/ubuntu/`

Variables

- GITLAB_ROOT_EMAIL: e-mail address for the initial GitLab root administrator account.
- GITLAB_ROOT_PASSWORD: password for the initial GitLab root administrator account.
- DOMAIN_NAME: fully qualified domain name where GitLab will be reachable, for example `gitlab.example.nl`.

Install `curl`. GitLab provides a repository setup script, and `curl` is needed to download it.

```
sudo apt install -y curl
```

Add the GitLab CE package repository. This makes the `gitlab-ce` package available through `apt`.

```
curl --location "https://packages.gitlab.com/install/repositories/gitlab/gitlab-ce/script.deb.sh" | sudo bash
```

Install GitLab CE. Before running the command, set the initial root e-mail address, root password, and public GitLab URL with the variables listed above.

```
sudo GITLAB_ROOT_EMAIL="<GITLAB_ROOT_EMAIL>" GITLAB_ROOT_PASSWORD="<GITLAB_ROOT_PASSWORD>" EXTERNAL_URL="https://<DOMAIN_NAME>/" apt install gitlab-ce
```

3.1.4 GitLab SMTP

Reference: `https://docs.gitlab.com/omnibus/settings/smtp/`

Configure SMTP so GitLab can send account, notification, and pipeline-related e-mails. Replace the example values with the mail server settings for the target environment.

Edit `/etc/gitlab/gitlab.rb`:

```
...

gitlab_rails['smtp_enable'] = true
gitlab_rails['smtp_address'] = "smtp.server"
gitlab_rails['smtp_port'] = 465
gitlab_rails['smtp_user_name'] = "smtp user"
gitlab_rails['smtp_password'] = "smtp password"
gitlab_rails['smtp_domain'] = "example.com"
gitlab_rails['smtp_authentication'] = "login"
gitlab_rails['smtp_enable_starttls_auto'] = true
gitlab_rails['smtp_openssl_verify_mode'] = 'peer'

# If your SMTP server does not like the default 'From: gitlab@localhost' you
# can change the 'From' with this setting.
gitlab_rails['gitlab_email_from'] = 'gitlab@example.com'
gitlab_rails['gitlab_email_reply_to'] = 'noreply@example.com'

# If your SMTP server is using a self signed certificate or a certificate which
# is signed by a CA which is not trusted by default, you can specify a custom ca file.
# Please note that the certificates from /etc/gitlab/trusted-certs/ are
# not used for the verification of the SMTP server certificate.
gitlab_rails['smtp_ca_file'] = '/path/to/your/cacert.pem'
```

Example configuration:

After saving the configuration file, reconfigure GitLab. This applies the SMTP settings and restarts the GitLab services that need the new configuration.

```
sudo gitlab-ctl reconfigure
```

3.1.5 GitLab Runner

Reference: `https://docs.gitlab.com/runner/install/linux-repository/`

Variables

- DOMAIN_NAME: fully qualified domain name of the GitLab instance that the runner registers with.
- RUNNER_TOKEN: registration token generated by GitLab when creating the instance runner.

Download the GitLab Runner repository setup script, inspect it, and then run it. This adds the official GitLab Runner package repository to the server.

```
curl -L "https://packages.gitlab.com/install/repositories/runner/gitlab-runner/script.deb.sh" -o script.deb.sh
less script.deb.sh
sudo bash script.deb.sh
```

Install GitLab Runner after the repository has been added.

```
sudo apt install gitlab-runner
```

Create a new instance runner in the GitLab web interface and copy the runner token. The token is needed when the runner is registered on the server. Follow the path below in the GitLab settings. Figure 1 shows the page where the token is generated.

Admin → CI/CD → Runners → Create instance runner

Register the runner on the GitLab server with the GitLab domain name and runner token from the variables above. This runner uses Docker as its executor and starts from the `alpine:latest` image by default.

```
sudo gitlab-runner register \
  --non-interactive \
  --url "https://<DOMAIN_NAME>/" \
  --token "<RUNNER_TOKEN>" \
  --executor "docker" \
  --docker-image alpine:latest \
  --description "docker-runner"
```

3.1.6 GitLab Container Registry

Reference: `https://docs.gitlab.com/administration/packages/container_registry/`

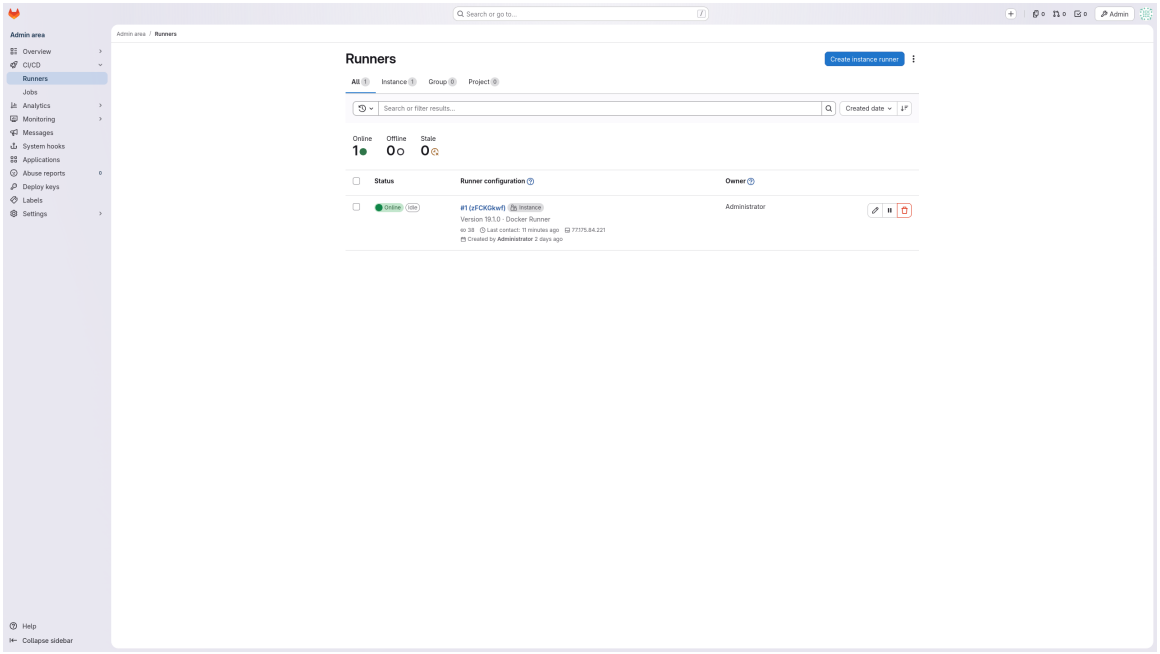

Figure 1: GitLab instance runner creation page.

This installation step still has to be documented before the container registry can be considered fully configured.

3.1.7 GitLab Pages

Reference: `https://docs.gitlab.com/user/project/pages/`

This installation step still has to be documented before GitLab Pages can be considered fully configured.

3.2 RKE2

3.2.1 Base Configuration, SSH & Firewall

Start on the RKE2 server by updating the package index and installing the available upgrades. This prepares the server for the Kubernetes and GPU-related packages.

```
sudo apt update && sudo apt upgrade -y
```

Enable and start SSH so administrators can manage the RKE2 server remotely.

```
sudo systemctl enable --now ssh.service
```

Open the ports required by RKE2, the Kubernetes API, etcd, node services, NodePort services, and the cluster networking components. Enable the firewall after the rules have been added.

```
sudo ufw allow 22/tcp
sudo ufw allow 6443/tcp
sudo ufw allow 9345/tcp
sudo ufw allow 10250/tcp
sudo ufw allow 2379/tcp
sudo ufw allow 2380/tcp
sudo ufw allow 2381/tcp
sudo ufw allow 30000:32767/tcp
sudo ufw allow 8472/udp
sudo ufw allow 9099/tcp
sudo ufw allow 51820/udp
sudo ufw allow 51821/udp
sudo ufw enable
```

3.2.2 NVIDIA Drivers

Reference: `https://docs.nvidia.com/datacenter/tesla/driver-installation-guide/ubuntu.html`

Install the Linux kernel headers for the currently running kernel. The NVIDIA driver build process needs these headers.

```
apt install linux-headers-$(uname -r)
```

Download and install the NVIDIA CUDA repository keyring, then refresh the package index. This makes the NVIDIA driver packages available through `apt`.

```
wget https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2404/
    x86_64/cuda-keyring_1.1-1_all.deb
sudo dpkg -i cuda-keyring_1.1-1_all.deb
sudo apt update
```

Install the NVIDIA compute and DKMS driver packages. DKMS rebuilds the kernel module when the kernel changes.

```
sudo apt -V install libnvidia-compute nvidia-dkms
```

Reboot the server after installing the driver. The reboot loads the NVIDIA kernel module and completes the driver installation.

```
sudo systemctl reboot
```

3.2.3 NVIDIA Container Toolkit

Reference: `https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install-guide.html`

Install the base tools needed to add and verify the NVIDIA Container Toolkit repository.

```
sudo apt-get update && sudo apt-get install -y --no-install-recommends \
  ca-certificates \
  curl \
  gnupg2
```

Add the NVIDIA Container Toolkit signing key and package repository. This repository provides the runtime integration for containers that need GPU access.

```
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey | sudo gpg --dearmor -o /usr/share/keyrings/nvidia-container-toolkit-keyring.gpg \
  && curl -s -L https://nvidia.github.io/libnvidia-container/stable/deb/nvidia-container-toolkit.list | \
  sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg] https://#g' | \
  sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list
```

Enable the experimental entries in the NVIDIA Container Toolkit repository file. This is needed when the selected package version is published through those entries.

```
sudo sed -i -e '/experimental/ s/^#//g' /etc/apt/sources.list.d/nvidia-container-toolkit.list
```

Refresh the package index so Ubuntu can read the NVIDIA Container Toolkit repository changes.

```
sudo apt update
```

Install the NVIDIA Container Toolkit packages at the selected version. Keeping the version explicit makes the installation reproducible.

```
export NVIDIA_CONTAINER_TOOLKIT_VERSION=1.19.1-1
sudo apt-get install -y \
  nvidia-container-toolkit=${NVIDIA_CONTAINER_TOOLKIT_VERSION} \
  nvidia-container-toolkit-base=${NVIDIA_CONTAINER_TOOLKIT_VERSION} \
  libnvidia-container-tools=${NVIDIA_CONTAINER_TOOLKIT_VERSION} \
  libnvidia-container1=${NVIDIA_CONTAINER_TOOLKIT_VERSION}
```

Configure containerd to use the NVIDIA runtime. This allows Kubernetes workloads to request GPU resources through the container runtime.

```
sudo nvidia-ctk runtime configure --runtime=containerd
```

3.2.4 Helm

Reference: `https://helm.sh/docs/intro/install`

Install Helm from the official package repository. Helm is used later to install Kubernetes charts, including the SMB CSI driver.

```
HELM_BUILDKITE_APT_KEY_ID="DDF78C3E6EBB2D2CC223C95C62BA89D07698DBC6"

sudo apt-get install curl gpg apt-transport-https --yes

curl -fsSL https://packages.buildkite.com/helm-linux/helm-debian/gpgkey > "${TMPDIR:-/tmp}/helm.gpg"

# Ensure that the key ID matches to prevent a repository compromise from establishing an attacker controlled key
if [ "$(gpg --show-keys --with-colons "${TMPDIR:-/tmp}/helm.gpg" | awk -F: '$1 == "fpr" {print $10}' | head -n 1)" != "${HELM_BUILDKITE_APT_KEY_ID}" ]; then echo "ERROR: Unexpected Helm APT key ID: potential key compromise"; exit 1; fi

cat "${TMPDIR:-/tmp}/helm.gpg" | gpg --dearmor | sudo tee /usr/share/keyrings/helm.gpg > /dev/null
echo "deb [signed-by=/usr/share/keyrings/helm.gpg] https://packages.buildkite.com/helm-linux/helm-debian/any/ any main" | sudo tee /etc/apt/sources.list.d/helm-stable-debian.list

sudo apt-get update
sudo apt-get install helm
```

3.2.5 RKE2

Reference: `https://docs.rke2.io/install/quickstart`

Variables

- PRIVATE_IP_ADDRESS: private network IP address of the RKE2 server that should be included in the cluster certificate.

Create the NetworkManager configuration directory. The next step places an RKE2-specific configuration file in this directory.

```
sudo mkdir -p /etc/NetworkManager/conf.d/
```

Configure NetworkManager so it does not manage interfaces created by the Kubernetes networking layer. This prevents conflicts with RKE2 networking interfaces such as flannel and calico interfaces.

Edit `/etc/NetworkManager/conf.d/rke2-canal.conf`:

```
[keyfile]
unmanaged-devices=interface-name:flannel*;interface-name:cali*;interface-
    name:tunl*;interface-name:vxlan.calico;interface-name:vxlan-v6.calico;
    interface-name:wireguard.cali;interface-name:wg-v6.cali
```

Restart networking so the NetworkManager configuration is applied.

```
sudo systemctl restart systemd-networkd
```

Create the RKE2 configuration directory. RKE2 reads its main configuration file from this location.

```
sudo mkdir -p /etc/rancher/rke2/
```

Create the RKE2 configuration file and set the node options. Replace `<PRIVATE_IP_ADDRESS>` with the private IP address that clients and other systems should use to reach this node.

Edit `/etc/rancher/rke2/config.yaml`:

```
write-kubeconfig-mode: "0644"
tls-san:
  - "<PRIVATE_IP_ADDRESS>"
node-label:
  - "role=gpu-node"
debug: true
```

Install RKE2 with the official installation script. This installs the RKE2 server binaries and systemd service files.

```
curl -sfL https://get.rke2.io | sudo sh -
```

Enable and start the RKE2 server service. The cluster starts after this command finishes.

```
sudo systemctl enable --now rke2-server.service
```

Add the RKE2 binaries to the shell path. This allows tools such as `kubectl` to run without the full binary path.

Edit `~/.bashrc`:

```
...

export PATH=$PATH:/var/lib/rancher/rke2/bin
```

Reload the shell configuration so the updated path is available in the current terminal session.

```
source .bashrc
```

Create the local kubeconfig directory and copy the RKE2 kubeconfig into it. This lets the current user manage the cluster with Kubernetes CLI tools.

```
mkdir -p ~/.kube
sudo cp /etc/rancher/rke2/rke2.yaml ~/.kube/config
```

3.2.6 GPU Operator

Reference: `https://docs.rke2.io/add-ons/gpu_operators`

Create the GPU Operator manifest in the RKE2 manifests directory. RKE2 automatically applies files from this directory, so the GPU Operator is deployed by the cluster.

Edit `/var/lib/rancher/rke2/server/manifests/gpu-operator.yaml`:

```
apiVersion: helm.cattle.io/v1
kind: HelmChart
metadata:
  name: gpu-operator
  namespace: kube-system
spec:
  repo: https://helm.ngc.nvidia.com/nvidia
  chart: gpu-operator
  version: v26.3.1
  targetNamespace: gpu-operator
  createNamespace: true
  valuesContent: |-
    cdi:
      nriPluginEnabled: true
```

Restart RKE2 after adding or changing the manifest. This lets the cluster reload the configuration and apply the GPU Operator deployment.

```
server: https://<RKE2_DOMAIN_NAME>:6443
sudo systemctl restart rke2-server
```

3.2.7 SMB CSI Driver

Reference: `https://github.com/kubernetes-csi/csi-driver-smb`

Install the SMB CSI driver into the Kubernetes cluster with Helm. This driver allows Kubernetes workloads to mount SMB shares as persistent storage.

```
helm repo add csi-driver-smb https://raw.githubusercontent.com/kubernetes-
    csi/csi-driver-smb/master/charts
helm install csi-driver-smb csi-driver-smb/csi-driver-smb --namespace kube-
    system --version 1.20.1
```

3.3 SMB

3.3.1 Base Configuration, SSH & Firewall

Start on the SMB server by updating the package index and installing the available upgrades.

```
sudo apt update && sudo apt upgrade -y
```

Enable and start SSH so administrators can manage the SMB server remotely.

```
sudo systemctl enable --now ssh.service
```

Open the ports for SSH and SMB file sharing, then enable the firewall.

```
sudo ufw allow 22/tcp
sudo ufw allow 445/tcp
sudo ufw enable
```

3.3.2 SMB

Reference: `https://ubuntu.com/server/docs/how-to/samba/file-server/`

Variables

- STUDY_FOLDER: name of the study directory that will be created under the Samba share.
- STUDY_USER: Linux and Samba user account that will be used to access the study share *min.8characters*.

Install Samba on the SMB server. Samba provides the file share used by the rest of the environment.

```
sudo apt update
sudo apt install -y samba
```

Create the study folder structure inside the Samba share. Replace `<STUDY_FOLDER>` with the folder name for the study.

```
sudo mkdir -p /srv/samba/<STUDY_FOLDER>/A_ShortDescription
sudo mkdir -p /srv/samba/<STUDY_FOLDER>/B_Documentation
sudo mkdir -p /srv/samba/<STUDY_FOLDER>/C_PersonalData
sudo mkdir -p /srv/samba/<STUDY_FOLDER>/D_DataPreparation
sudo mkdir -p /srv/samba/<STUDY_FOLDER>/E_ResearchData
sudo mkdir -p /srv/samba/<STUDY_FOLDER>/F_DataAnalysis
sudo mkdir -p /srv/samba/<STUDY_FOLDER>/G_Output
sudo mkdir -p /srv/samba/<STUDY_FOLDER>/H_Hidden
```

Set ownership on the study folder so the Linux study user can manage it. This keeps the folder structure easy to maintain before access is handled through Samba.

```
sudo chown -R <STUDY_USER>:<STUDY_USER> /srv/samba/<STUDY_FOLDER>/
```

Configure the Samba share. The example exposes the Samba directory as a writable share.

Edit `/etc/samba/smb.conf`:

```
...

[sambashare]
   comment = Samba RFS folder
   path = /srv/samba/
   browsable = yes
   read only = no
   create mask = 0755
   directory mask = 0755
```

Restart the Samba services so the new share configuration is applied.

```
sudo systemctl restart smbd.service nmbd.service
```

3.4 Connections

3.4.1 GitLab to RKE2

Variables

- DEPLOY_TOKEN_USERNAME: username generated for the GitLab deploy token.
- DEPLOY_TOKEN_PASSWORD: password or token value generated for the GitLab deploy token.

Use the deploy token created during the templates setup. On RKE2, configure registry authentication so the cluster can pull container images from the GitLab registry.

Edit `/etc/rancher/rke2/registries.yaml`:

```
configs:
  "<DOMAIN_NAME>:5050":
    auth:
      username: "<DEPLOY_TOKEN_USERNAME>"
      password: "<DEPLOY_TOKEN_PASSWORD>"
```

Restart RKE2 so it reloads the registry authentication configuration.

```
sudo systemctl restart rke2-server
```

3.4.2 RKE2 to GitLab

Variables

- PRIVATE_IP_ADDRESS: private network IP address that GitLab Runner should use to reach the RKE2 API server.

On RKE2, copy the kubeconfig from `/etc/rancher/rke2/rke2.yaml`. This file contains the cluster connection information that GitLab Runner needs.

On GitLab, create the kubeconfig file for the GitLab Runner user. First create the file, then paste the copied kubeconfig content into it.

```
sudo -u gitlab-runner mkdir -p /home/gitlab-runner/.kube
sudo -u gitlab-runner touch /home/gitlab-runner/.kube/config
```

Paste the copied RKE2 kubeconfig content into `/home/gitlab-runner/.kube/config`.

Update the kubeconfig so it points to the private RKE2 address. This makes GitLab Runner connect to the cluster through the internal network.

Edit `/home/gitlab-runner/.kube/config`:

```
...
server: https://<PRIVATE_IP_ADDRESS>:6443
...
```

Mount the kubeconfig into the GitLab Runner container and enable privileged mode. This gives CI jobs access to the Kubernetes configuration when they deploy to RKE2.

Edit `/etc/gitlab-runner/config.toml`:

```
...
volumes = ["/cache", "/home/gitlab-runner/.kube:/root/.kube:ro"]
privileged = true
...
```

Restart GitLab Runner after changing its configuration. The runner will then use the updated mount and privilege settings.

```
sudo gitlab-runner restart
```

3.4.3 GitLab and RKE2 to SMB

Variables

- SMB_USERNAME: Samba username used by GitLab jobs or Kubernetes workloads to access the SMB share.

- SMB_PASSWORD: Samba password for the SMB user account.

On GitLab, add `SMB_USERNAME` and `SMB_PASSWORD` as masked and hidden repository variables for each project that needs the SMB share. Follow this path in the GitLab project settings:

Projects → `<PROJECT>` → Settings → CI/CD → Variables → CI/CD Variables

3.5 Templates

The templates repository is the shared GitLab project that contains the reusable CI/CD, Docker, and Kubernetes files for user repositories. Create this project inside the same GitLab group as the user projects. In this environment, the templates are expected at:

```
umcu-ai-deployments/templates
```

The project path matters because user projects include the shared pipeline from this location. The shared pipeline also clones the templates repository during the build and deploy jobs. If another project path is used, update the include paths and clone paths in the CI templates before users start creating projects.

3.5.1 Group Settings

Variables

- GROUP: GitLab group that contains the templates repository, user projects, and container images.
- DEPLOY_TOKEN_USERNAME: username generated for the GitLab deploy token.
- DEPLOY_TOKEN_PASSWORD: password or token value generated for the GitLab deploy token.
- SMB_SOURCE: SMB share source used by the Kubernetes SMB CSI volume.

Create a deploy token for the group that contains the user projects and container images. RKE2 uses this token to authenticate to the GitLab container registry when Kubernetes pulls project images. Follow the path below in the GitLab group settings. Figure 2 shows the group deploy token page.

Groups → `<GROUP>` → Settings → Repository → Deploy tokens

Add `SMB_SOURCE` as a group CI/CD variable for the same parent group. Set the value to the full SMB source path, for example `//<PRIVATE_IP_ADDRESS>/samba`. Replace `<PRIVATE_IP_ADDRESS>` with the private IP address of the SMB server. Follow the path below in the GitLab group settings. Figure 3 shows the group variables page.

Groups → `<GROUP>` → Settings → CI/CD → Variables → CI/CD Variables

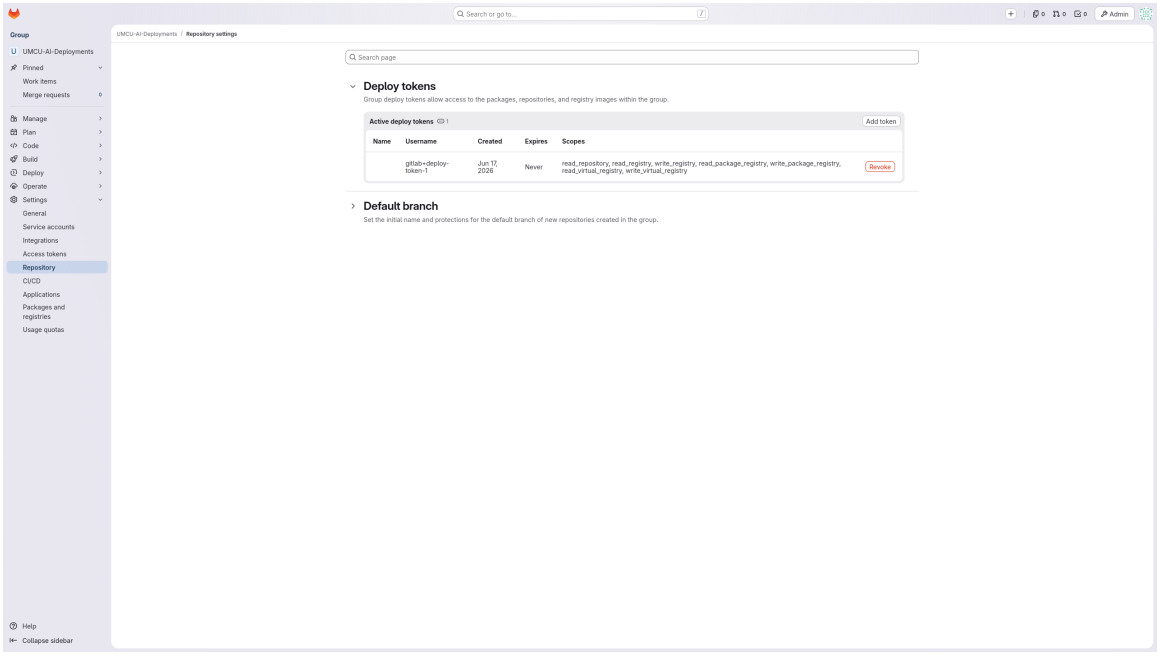

Figure 2: GitLab group deploy token settings.

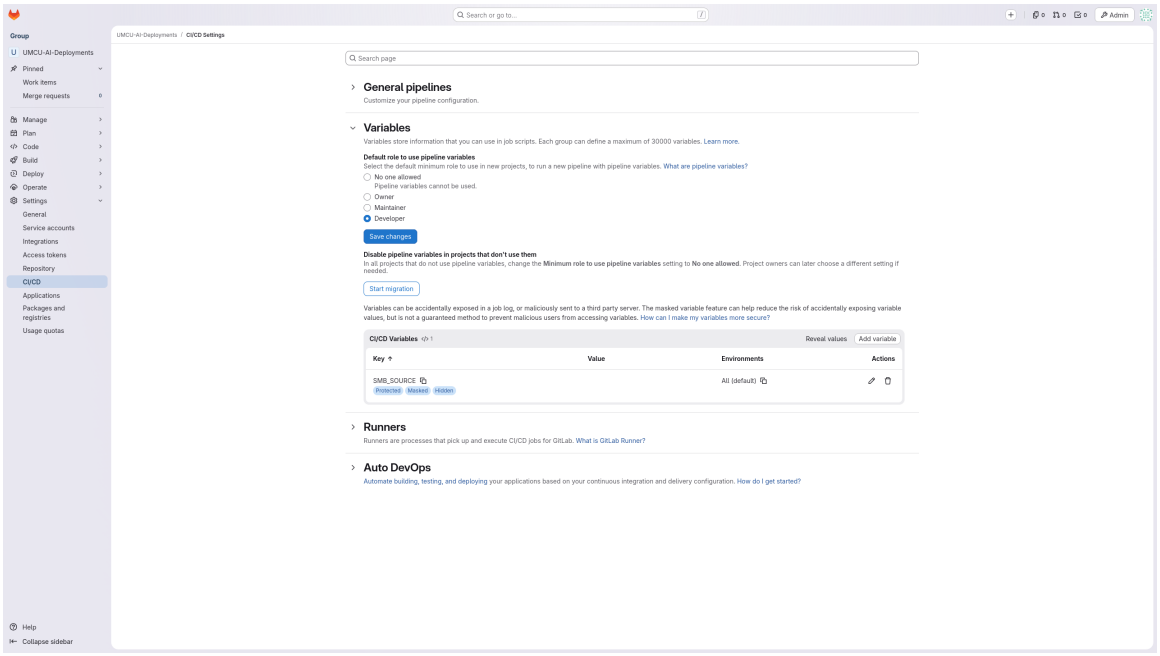

Figure 3: GitLab group CI/CD variables page.

3.5.2 Project Permissions

Allow the templates repository to be accessed by CI/CD job tokens from the full group. This lets pipelines in the group read the shared templates repository when they include the common CI/CD templates. Follow the path below in the GitLab project settings. Figure 4 shows the job token allowlist with the deployment group added.

Projects → `<TEMPLATES_PROJECT>` → Settings → CI/CD → Job token permissions

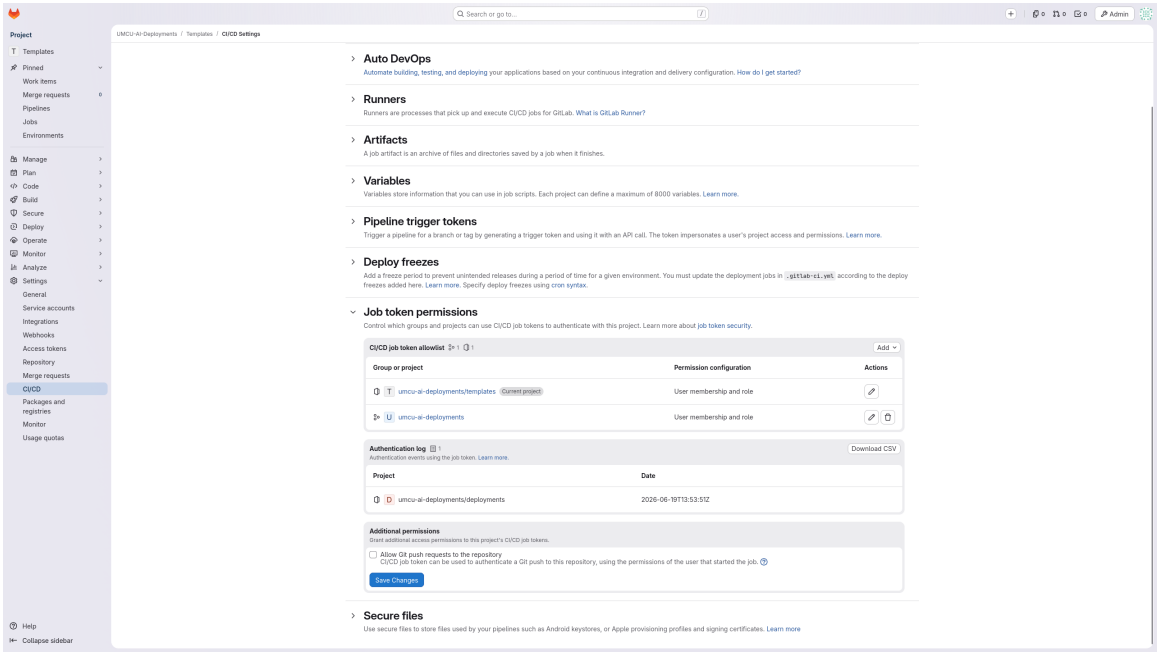

Figure 4: Templates repository job token permissions with the deployment group in the CI/CD job token allowlist.

The templates repository should contain the following structure. Figure 5 shows the same repository in GitLab after the shared folders have been pushed.

```
.
|-- gitlab
|   |-- shared
|   |   |-- .gitlab-ci.yml
|   |-- tensorflow-gpu-environment
|   |   |-- .gitlab-ci.yml
|   |-- python-gpu-environment
|   |   |-- .gitlab-ci.yml
|   |-- docker-gpu-environment
|       |-- .gitlab-ci.yml
|-- docker
|   |-- tensorflow-gpu-environment
```

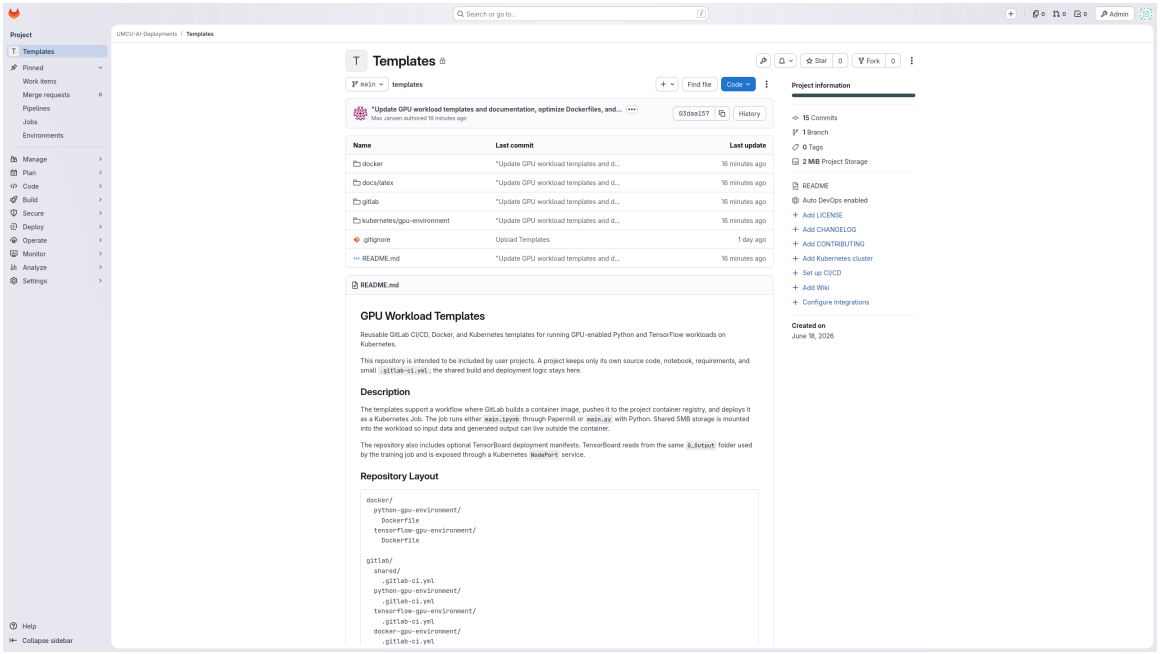

Figure 5: Templates repository in GitLab with the shared CI/CD, Docker, and Kubernetes folders.

```
|    |    |-- Dockerfile
|    |-- python-gpu-environment
|         |-- Dockerfile
|-- kubernetes
     |-- gpu-environment
          |-- deployment.yml
          |-- job.yml
          |-- pv.yml
          |-- pvc.yml
          |-- service.yml
```

The files in `gitlab` define the CI templates that user projects include. The environment-specific templates mainly select the Dockerfile path. After that, they include `gitlab/shared/.gitlab-ci.yml`, which contains the shared build and deployment pipeline.

The files in `docker` define the standard GPU container images. The TensorFlow template uses a TensorFlow GPU Dockerfile that already includes the common notebook and TensorFlow packages. The Python template uses a Python GPU Dockerfile and installs dependencies from the user project's `requirements.txt`. The Docker template does not use a shared Dockerfile; it expects the user project to provide its own `Dockerfile`.

The files in `kubernetes/gpu-environment` define the Kubernetes resources applied by the shared pipeline. From these manifests, the pipeline creates or updates the SMB-backed persistent volume, persistent volume claim, training job, and TensorBoard service.

After pushing the templates repository to GitLab, verify that user project pipelines can read it with their CI/CD job token. This permission is configured in the templates project settings under job token permissions. Without it, user pipelines may start but fail when they try to include or clone the templates repository.

Finally, review the values that are specific to the environment before users start using the templates. The GitLab project path in the CI files must match the templates repository location. The Kubernetes storage manifests must point to the correct SMB share and study folder layout. The Dockerfiles should use a CUDA base image version that matches the RKE2 GPU runtime.

4 Manual

4.1 Before You Start

This manual is a follow-along guide for creating a user project that runs Python or notebook code on the GPU-enabled RKE2 environment. Read it once before starting. Some choices, especially the template and storage layout, affect files that are added later in the workflow.

The user repository stays small. It contains the project code and a short GitLab CI configuration. The shared templates handle the container build, container registry upload, Kubernetes deployment, SMB storage mount, and optional TensorBoard access.

4.2 Templates Repository

The Templates repository contains the reusable CI/CD and deployment logic. User projects do not copy this logic into their own repository. Instead, they include one template file from the Templates repository in `.gitlab-ci.yml`.

The Templates repository has three main parts. The `gitlab` folder contains the CI templates that user projects include. The `docker` folder contains the standard Dockerfiles for the TensorFlow and Python GPU templates. The `kubernetes` folder contains the Kubernetes manifests that create the Job, storage resources, and TensorBoard service.

This design keeps user projects focused on the actual work: the model or processing code, optional dependency files, and the template include. When the pipeline starts, GitLab reads the selected template from the Templates repository and uses it to build and deploy the project.

4.3 Step 1: Create the GitLab Project

Create a new GitLab project inside the group that is allowed to use the shared templates repository. The project must be created inside this group because the pipeline reads the templates with the GitLab CI job token. If the project is created somewhere else, the include can fail or the pipeline may not be allowed to clone the shared templates. Figure 6 shows the GitLab blank project form with the deployment group selected.

After creating the project, clone it locally or open it in the GitLab web editor. The remaining steps are performed in the root of this new repository. Figure 7 shows an example user repository with the expected files.

4.4 Step 2: Add the Entry Point

Add the code that should run in the GPU environment. The pipeline looks for one of the following files in the repository root:

- `main.ipynb`: executed with Papermill.
- `main.py`: executed with Python.

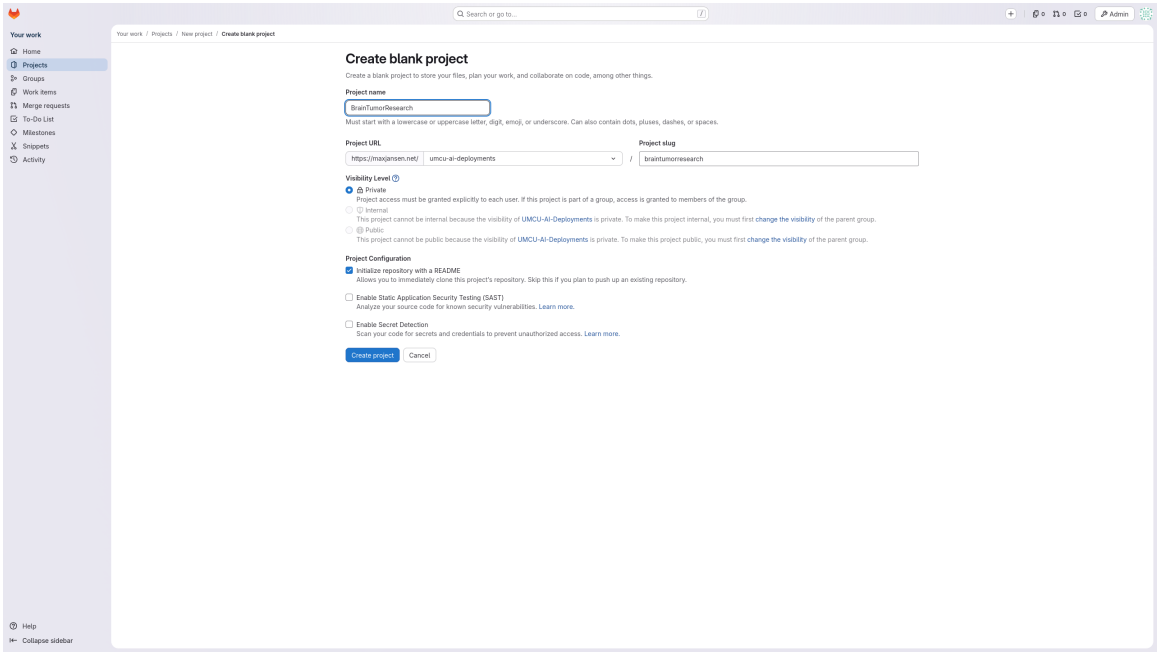

Figure 6: GitLab blank project creation form for a new user repository.

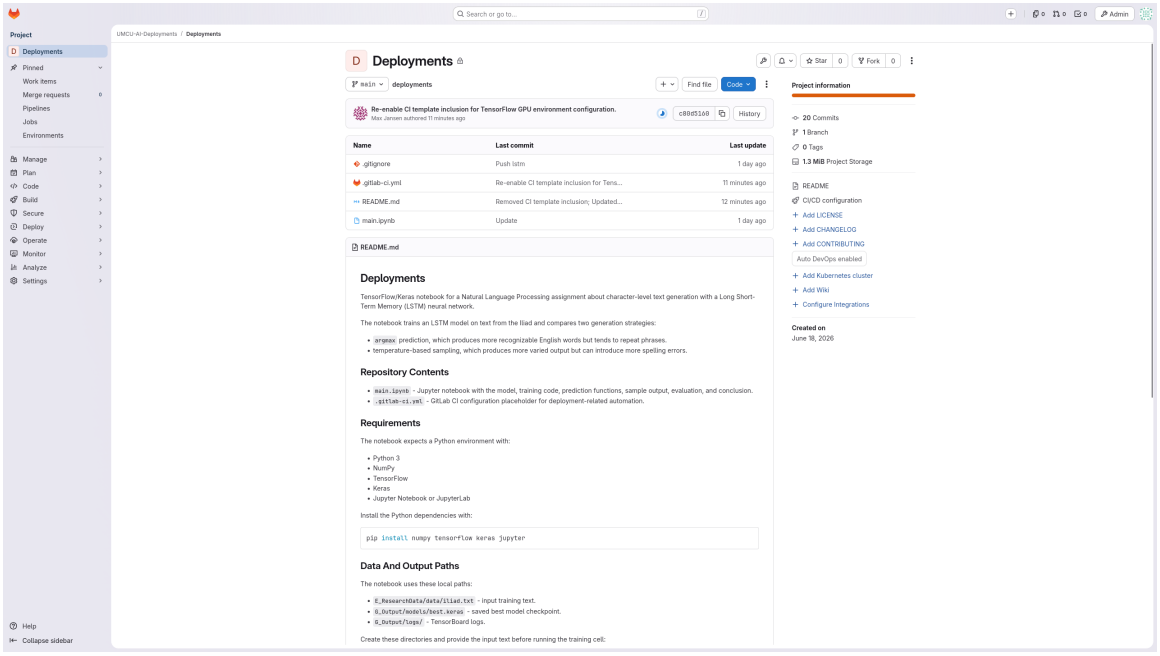

Figure 7: Example user repository with a notebook entry point and GitLab CI configuration.

Only one entry point is required. If both files exist, the notebook is executed first because the templates check for `main.ipynb` before `main.py`.

Project code should use the mounted storage folders directly:

- Read input data from `E_ResearchData`. This folder is mounted read-only.
- Write generated output to `G_Output`.
- For notebook projects, the executed notebook is written under `G_Output/notebooks`.

4.5 Step 3: Choose a Template

Choose one of the available templates before creating `.gitlab-ci.yml`. Start with the simplest template that fits the project.

4.5.1 `tensorflow-gpu-environment`

Choose `tensorflow-gpu-environment` when the project can run with the standard TensorFlow GPU image. This is the easiest option for notebook-based experiments and TensorFlow workloads because the image already includes:

- `jupyter`
- `papermill`
- `pandas`
- `numpy`
- `matplotlib`
- `tensorflow`
- `tensorboard`

With this template, the user normally only needs to add `main.ipynb` or `main.py`. Extra project files can be placed next to the entry point. They are copied into the container image during the build.

4.5.2 `python-gpu-environment`

Choose `python-gpu-environment` when the project needs extra Python packages but does not need a custom Dockerfile. Add a `requirements.txt` file to the repository root. The image installs the packages from that file before it copies the project code.

For notebook projects, add `papermill` to `requirements.txt`. The Kubernetes Job uses Papermill to execute `main.ipynb`. The requirements file must also include the other notebook execution dependencies that the code needs.

4.5.3 `docker-gpu-environment`

Choose `docker-gpu-environment` only when the project needs custom system packages, a different framework stack, or a dependency installation process that does not fit in `requirements.txt`. In this case, add a `Dockerfile` to the repository root. The shared pipeline still builds and deploys the image, but the project is responsible for installing the runtime, Python packages, system packages, and tools required by the entry point.

Base custom Dockerfiles on the same NVIDIA CUDA image used by the other GPU templates whenever possible. This keeps the CUDA and cuDNN versions aligned with the RKE2 GPU runtime and reduces the chance of driver, library, or TensorFlow compatibility problems.

```
FROM nvidia/cuda:12.9.2-cudnn-devel-ubuntu24.04
```

A minimal Dockerfile for a notebook-capable project can look like this:

```
FROM nvidia/cuda:12.9.2-cudnn-devel-ubuntu24.04

RUN apt-get update && apt-get install -y --no-install-recommends \
    python3 \
    python3-pip \
    libgl1 \
    && rm -rf /var/lib/apt/lists/*

RUN pip3 install --no-cache-dir --break-system-packages \
    papermill \
    pandas \
    numpy \
    matplotlib

WORKDIR /app
COPY . .
```

For notebook projects, `papermill` must be available in the Docker image together with a usable notebook kernel. For Python projects, make sure the image can run `python3 main.py`. The Dockerfile does not need to contain the full pipeline logic. If the image defines its own `CMD`, the Kubernetes Job command from the shared template overrides it during deployment.

4.6 Step 4: Add the CI Configuration

Add a file named `.gitlab-ci.yml` to the root of the repository. This file includes the selected shared template.

```
include:
  - project: "<GROUP>/templates"
    file: "gitlab/<TEMPLATE>/.gitlab-ci.yml"
```

For the current environment, a TensorFlow GPU project uses:

```
include:
  - project: "umcu-ai-deployments/templates"
```

```
    file: "gitlab/tensorflow-gpu-environment/.gitlab-ci.yml"
```

Replace the template path when using `python-gpu-environment` or `docker-gpu-environment`.

4.7 Step 5: Check the Repository Layout

Before pushing, check that the repository layout matches the selected template.

A minimal TensorFlow notebook project has the following structure:

```
.
|-- .gitlab-ci.yml
|-- main.ipynb
```

A Python project with custom dependencies has the following structure:

```
.
|-- .gitlab-ci.yml
|-- main.py
|-- requirements.txt
```

A project with a custom container image has the following structure:

```
.
|-- .gitlab-ci.yml
|-- Dockerfile
|-- main.py
```

Additional source files, modules, configuration files, and helper scripts can be added to the repository. They are copied into the container image and are available from the application working directory during execution.

4.8 Step 6: Configure Project Variables

The pipeline needs SMB credentials so Kubernetes can mount the shared storage. Add these CI/CD variables to the GitLab project:

- `SMB_USERNAME`: username for the Samba account.
- `SMB_PASSWORD`: password for the Samba account.
- `STUDY_FOLDER`: name of the study directory on the SMB share.

Store these values as protected, masked, and hidden variables when possible. Configure them as GitLab CI/CD variables. Use project-level variables when only one project needs access, or group-level variables when multiple projects use the same storage account. Figure 8 shows the project variables page.

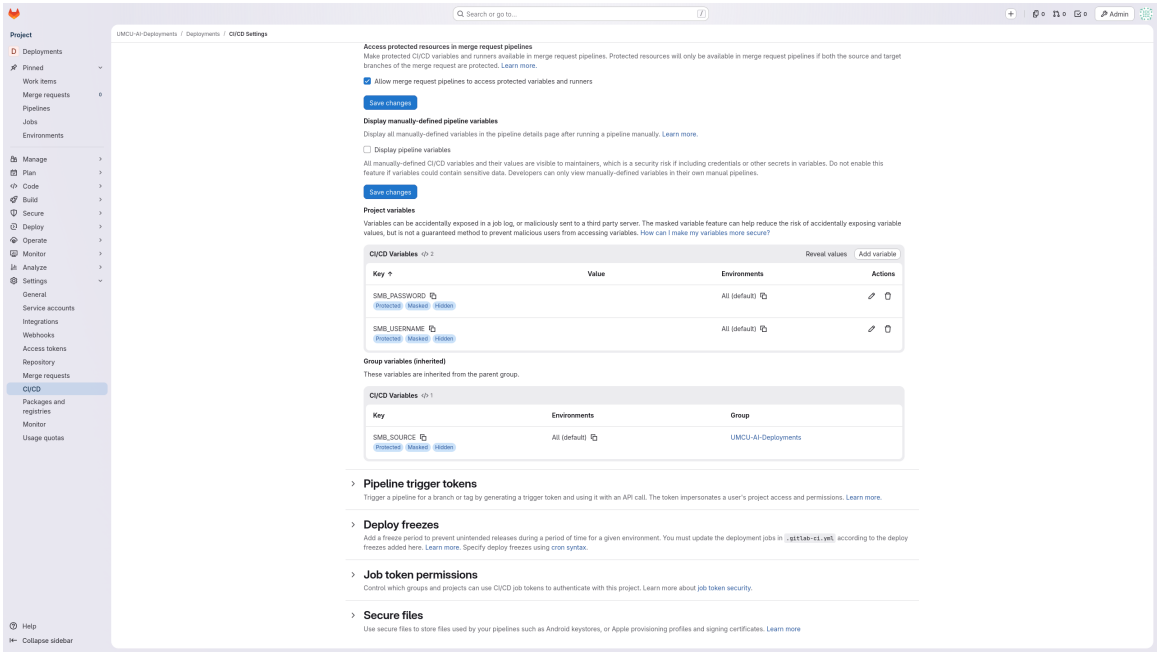

Figure 8: Project CI/CD variables used by the deployment pipeline.

Table 1 Content of the different folders

Folder	Description of content of the folders
A_ShortDescription	A short description of the study. You can suffice here with the abstract.
B_Documentation	All documentation relevant to the project is stored here and kept up to date during the whole project (e.g. METC documents, ISF)
C_PersonalData	Contains **directly identifying personal data**, including CodeLists (study_ID vs patient_ID) and source data with personal data. Only researchers with authorization to read personal data have access to this folder (e.g. based on informed consents)
D_DataPreparation	Location where the data manager stores personal data and linkage scripts and pseudonymization keys. Data are (technically cleansed and) pseudonymized during the data gathering and processing phase.
E_ResearchData	Contains **pseudonymized data** and metadata, e.g. pseudonymized source data. For every study it is mandatory to store a frozen data set to create a base for the analysis stage (authoritative source).
F_DataAnalysis	Contains all scripts, data and documents essential for performing the data analysis.
G_Output	All publications, articles, data and other deliverables.
H_Hidden	Contains information that requires limited access for selected team member(s), e.g. blinded randomization key, confidential contracts, files only meant for specific persons.

Figure 9: Expected SMB study folder structure.

4.9 Step 7: Verify the Storage Paths

Before starting the pipeline, make sure the study folder exists on the SMB share and contains the expected folder structure. Figure 9 shows the expected SMB folder layout.

- `E_ResearchData`: input data for the model or processing script.
- `G_Output`: output location for generated files, model results, logs, and executed notebooks.

The current Kubernetes template maps these folders from the configured study folder on the SMB share. Before running a project for another study folder, make sure the Kubernetes storage template points to the correct SMB subpaths and that the corresponding folders exist on the SMB server.

4.10 Step 8: Commit and Push

Commit the project files and push them to GitLab. GitLab starts the pipeline automatically after the push.

The pipeline runs in two main stages:

- `build`: builds the container image and pushes it to the GitLab container registry.
- `deploy`: creates or updates the Kubernetes storage resources and starts the Kubernetes Job.

4.11 Step 9: Monitor the Pipeline

Open the project pipeline page in GitLab and watch the status. A successful pipeline means that the image was built, the Kubernetes resources were applied, and the training or processing job was submitted. Figure 10 shows the pipeline view with the build, deploy, and log jobs.

If the pipeline fails, open the failed job log first. Build failures usually point to missing dependencies or Dockerfile problems. Deploy failures usually point to Kubernetes access, registry authentication, or SMB credential problems. Figure 11 shows the job log view used for troubleshooting.

TensorBoard can be used when the project writes TensorBoard event files to `G_Output`. When the TensorBoard job is enabled in the pipeline, GitLab creates an environment with a URL that opens the TensorBoard service. Open the project environments page in GitLab and select the TensorBoard environment for the latest pipeline. Figure 12 shows the environment link, and Figure 13 shows the TensorBoard dashboard after it opens.

4.12 Step 10: Check the Results

After the run completes, check the study folder on the SMB share. Input data should remain in `E_ResearchData`. Generated results, plots, model files, logs, and executed notebooks should be stored in `G_Output`.

For notebook projects, Papermill writes the executed notebook to `G_Output/notebooks`. Use the pipeline and Kubernetes logs to confirm whether the entry point completed successfully.

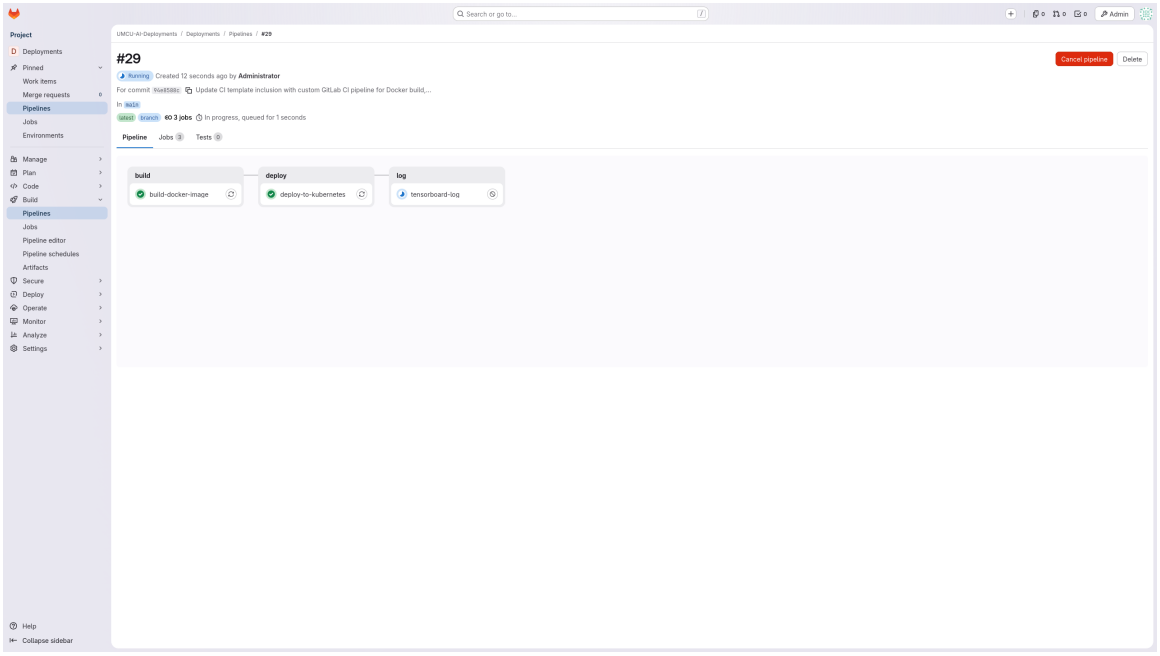

Figure 10: GitLab pipeline page showing the build, deploy, and TensorBoard log jobs.

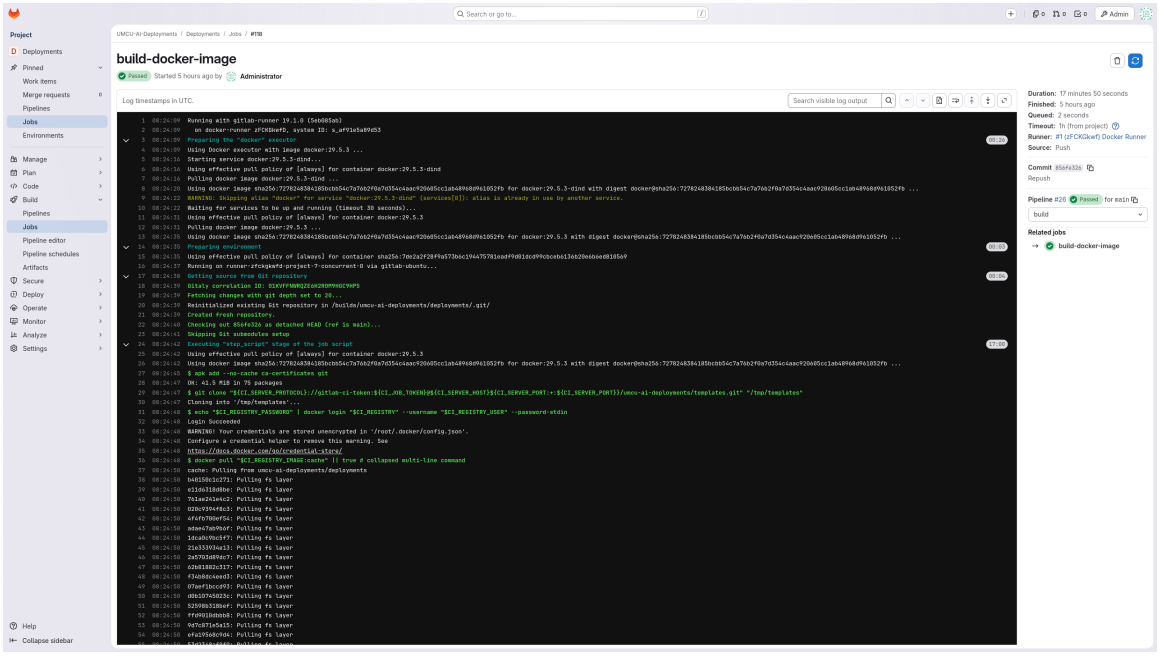

Figure 11: GitLab job page used to inspect job logs and pipeline output.

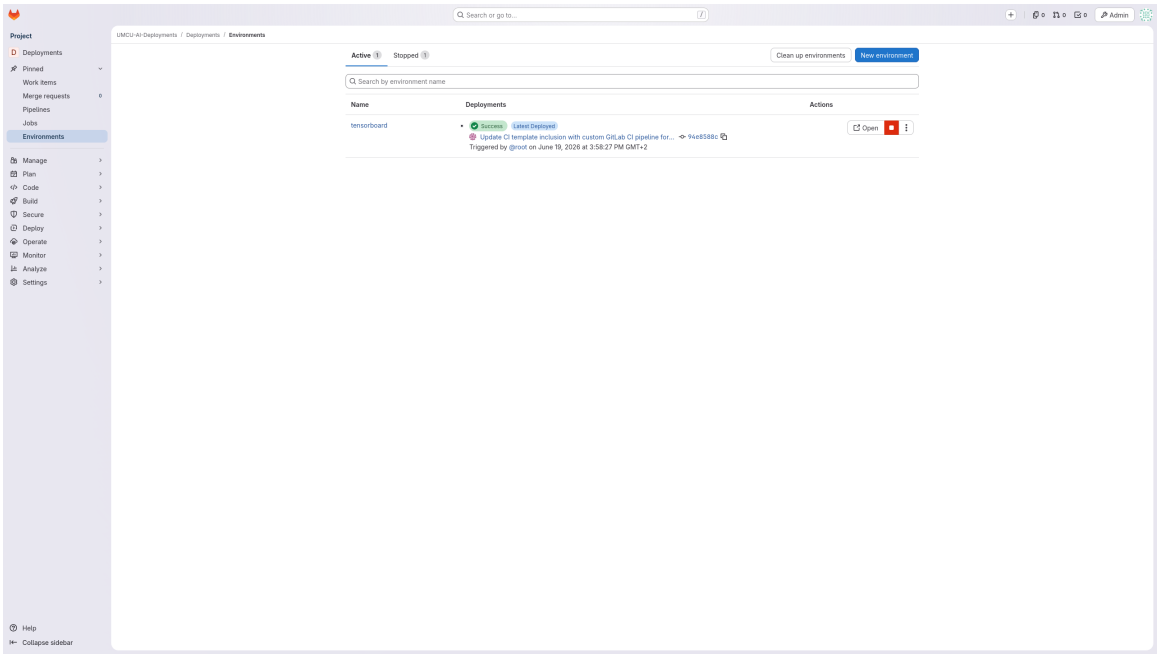

Figure 12: GitLab environments page with the TensorBoard environment link.

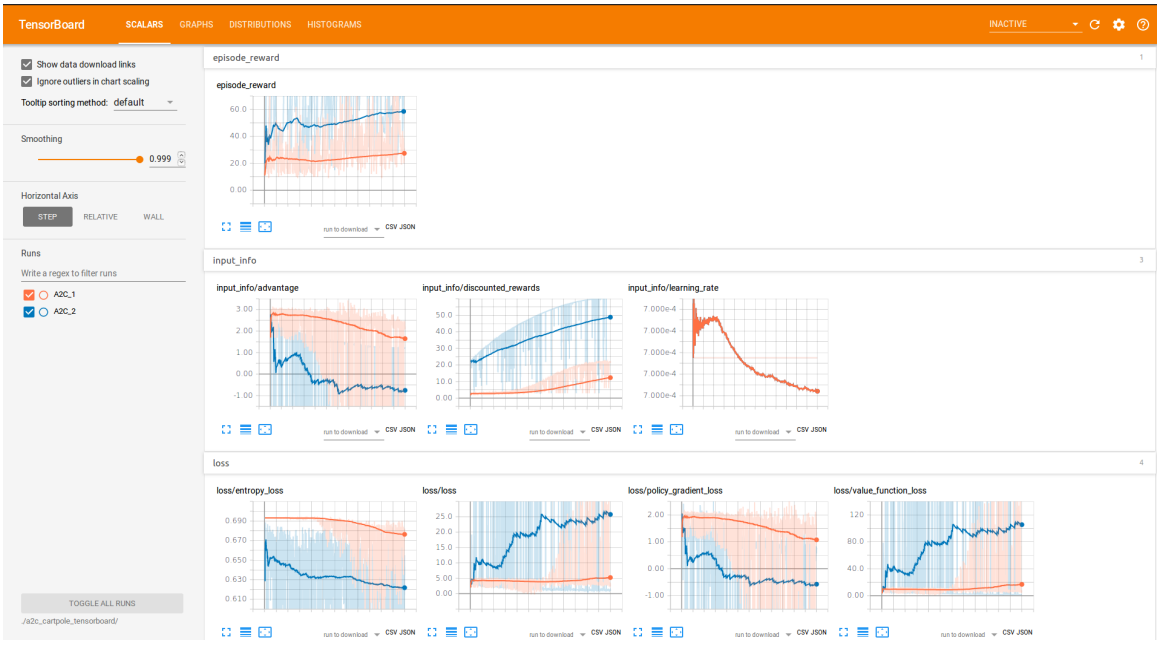

Figure 13: TensorBoard dashboard opened from the GitLab environment URL.

4.13 Final Checklist

- The project is created inside the GitLab group that has access to the shared templates.
- The repository contains `main.ipynb` or `main.py` at the root.
- The repository contains `.gitlab-ci.yml` with the correct shared template include.
- The selected template matches the project requirements.
- `requirements.txt` is present when using `python-gpu-environment`.
- `Dockerfile` is present when using `docker-gpu-environment`.
- Notebook projects include `papermill` when using the Python or Docker template.
- The project reads input data from `E_ResearchData`.
- The project writes output data to `G_Output`.
- `SMB_USERNAME` and `SMB_PASSWORD` are configured as CI/CD variables.

5 References

The following URLs are referenced throughout this document. They are collected here so the original documentation and download locations can be found quickly.

5.1 Documentation

- https://docs.docker.com/engine/install/ubuntu/
- https://docs.gitlab.com/install/package/ubuntu/
- https://docs.gitlab.com/omnibus/settings/smtp/
- https://docs.gitlab.com/runner/install/linux-repository/
- https://docs.gitlab.com/administration/packages/container_registry/
- https://docs.gitlab.com/user/project/pages/
- https://docs.nvidia.com/datacenter/tesla/driver-installation-guide/ubuntu.html
- https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install-guide.html
- https://helm.sh/docs/intro/install
- https://docs.rke2.io/install/quickstart
- https://docs.rke2.io/add-ons/gpu_operators
- https://github.com/kubernetes-csi/csi-driver-smb
- https://ubuntu.com/server/docs/how-to/samba/file-server/

5.2 Repositories and Downloads

- https://download.docker.com/linux/ubuntu/gpg
- https://download.docker.com/linux/ubuntu
- https://packages.gitlab.com/install/repositories/gitlab/gitlab-ce/script.deb.sh
- https://packages.gitlab.com/install/repositories/runner/gitlab-runner/script.deb.sh
- https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2404/x86_64/cuda-keyring_1.1-1_all.deb
- https://nvidia.github.io/libnvidia-container/gpgkey
- https://nvidia.github.io/libnvidia-container/stable/deb/nvidia-container-toolkit.list

- https://packages.buildkite.com/helm-linux/helm-debian/gpgkey
- https://packages.buildkite.com/helm-linux/helm-debian/any/
- https://get.rke2.io
- https://helm.ngc.nvidia.com/nvidia
- https://raw.githubusercontent.com/kubernetes-csi/csi-driver-smb/master/charts